Building and Securing a REST API

MoMo SMS Data Processing System

Team Nerds

Name	Role	Contribution
Nziza Samuel	Team Lead	Data Parsing - XML to JSON
Lorris Hira	Backend Dev	API Server - CRUD Endpoints
Bruce Manzi	Security	Authentication & Security
Uwase Huguette	DSA	Search Algorithms & Analysis
A. Irakarama	Documentation	API Docs, Testing & Screenshots

---

1. Introduction to API Security

Overview

This project implements a REST API on top of the MoMo SMS data processing pipeline. The API allows external clients such as mobile apps and web applications to securely access, create, update, and delete transaction records through standard HTTP methods. Security is enforced using HTTP Basic Authentication on every endpoint.

What is Basic Authentication?

Basic Authentication is an HTTP authentication scheme where the client encodes the username and password in Base64 and sends them in the Authorization header with every request. The server decodes and validates these credentials before processing the request.

Example Authorization header:

```
Authorization: Basic YWRtaW46cGFzc3dvcmQxMjM=
```

Limitations of Basic Authentication

Basic Auth has several serious weaknesses that make it unsuitable for production systems:

Weakness	Explanation
Credentials sent every request	Username and password are transmitted with every HTTP request, increasing exposure.
Base64 is not encryption	Base64 can be decoded instantly by anyone who intercepts the header — it provides no s
No token expiry	There is no mechanism to expire credentials. Once compromised, they remain valid indefi
Vulnerable to replay attacks	An intercepted header can be reused by an attacker to make valid requests.
No fine-grained permissions	All users share the same credentials — there is no role-based access control.

Stronger Alternatives

JWT (JSON Web Tokens): After login, the server issues a signed token containing user claims and an expiry time. The client sends this token in subsequent requests. Since tokens are stateless and time-limited, they are far more secure than Basic Auth. If a token is compromised, it expires automatically.

OAuth 2.0: An industry-standard authorization framework that allows users to grant limited access to their resources without sharing credentials. Used by Google, Facebook, and GitHub. Supports scopes, refresh tokens, and revocation — making it suitable for multi-client production APIs.

2. API Endpoint Documentation

Base URL: http://localhost:8080 | Authentication: Basic Auth (username: admin, password: password123)

Method	Endpoint	Description	Auth Required
GET	/transactions	List all transactions (supports limit & offset)	Yes
GET	/transactions/{id}	Get a single transaction by ID	Yes
POST	/transactions	Create a new transaction	Yes
PUT	/transactions/{id}	Update an existing transaction	Yes
DELETE	/transactions/{id}	Delete a transaction	Yes

GET /transactions

Request:

```
curl -u admin:password123 http://localhost:8080/transactions
```

Response (200 OK):

```
{ "total": 1691, "limit": 1691, "offset": 0, "transactions": [ { "id": 1,
"transaction_type": "incoming", "amount": 2000.0, "sender": "Jane Smith", ... } ] }
```

GET /transactions/{id}

Request:

```
curl -u admin:password123 http://localhost:8080/transactions/1
```

Response (200 OK):

```
{ "id": 1, "transaction_type": "incoming", "amount": 2000.0, "sender": "Jane Smith",
"receiver": null, "timestamp": "2024-05-10 16:30:51", "balance": 2000.0, "fee": null }
```

POST /transactions

Request:

```
curl -u admin:password123 -X POST http://localhost:8080/transactions \ -H
"Content-Type: application/json" \ -d
'{"transaction_type":"incoming","amount":5000,"sender":"Test User"}'
```

Response (201 Created):

```
{ "message": "Transaction created.", "transaction": { "id": 1692, "transaction_type":
"incoming", "amount": 5000 } }
```

PUT /transactions/{id}

Request:

```
curl -u admin:password123 -X PUT http://localhost:8080/transactions/1 \ -H
"Content-Type: application/json" \ -d '{"amount": 9999}'
```

Response (200 OK):

```
{ "message": "Transaction updated.", "transaction": { "id": 1, "amount": 9999, ... } }
```

DELETE /transactions/{id}

Request:

```
curl -u admin:password123 -X DELETE http://localhost:8080/transactions/1691
```

Response (200 OK):

```
{ "message": "Transaction 1691 deleted." }
```

Error Codes

Code	Meaning	When it occurs
200	OK	Request succeeded
201	Created	New transaction successfully added
400	Bad Request	Invalid JSON body or non-numeric ID
401	Unauthorized	Missing or wrong credentials
404	Not Found	Transaction ID does not exist or wrong endpoint

3. DSA Comparison Results

Huguette implemented and benchmarked two search algorithms on a dataset of 25 MoMo transaction records. Each algorithm was run 10,000 times per ID to produce reliable average times in microseconds.

Algorithms Compared

Algorithm	Data Structure	Time Complexity	How it works
Linear Search	List	O(n)	Scans every record one by one until the target ID is found.
Dictionary Lookup	Hash Table (dict)	O(1) average	Hashes the key directly to its memory location — no scannin

Sample Benchmark Results

The table below shows representative timing results from the benchmark run on 25 records:

Transaction ID	Linear Search (us)	Dict Lookup (us)	Speedup
1 (best case)	~0.25	~0.08	~3x
13 (mid list)	~0.85	~0.08	~10x
25 (worst case)	~1.60	~0.08	~20x
Average (all 25)	~0.90	~0.08	~11x

Why is Dictionary Lookup Faster?

Linear search has O(n) time complexity — in the worst case (searching for the last element or a missing ID), it must inspect every single record in the list. With 25 records, that is up to 25 comparisons. As the dataset grows to 1,691 records (our full MoMo dataset), the worst case becomes 1,691 comparisons.

Dictionary lookup has O(1) average time complexity. Python's dict is backed by a hash table. When you call transactions_dict[id], Python computes a hash of the key and goes directly to the memory slot — no scanning required. The time is constant regardless of dataset size.

Other Data Structures to Consider

Structure / Algorithm	Complexity	Best Use Case
Binary Search (sorted list)	O(log n)	When a dict is unavailable; list must be sorted by ID.
B-Tree / SQL Index	O(log n)	Database-level search; used by PostgreSQL index on id column.
Trie / Inverted Index	O(k)	Searching by sender name or partial text matching.

4. Reflection on Basic Auth Limitations

Basic Authentication was implemented in this project as required by the assignment. However, it is important to understand why it should never be used in a real production API and what better alternatives exist.

Why Basic Auth is Weak

The fundamental problem with Basic Auth is that it transmits credentials with every single request. Even though the credentials are Base64-encoded, Base64 is an encoding scheme — not encryption. Anyone who intercepts the HTTP request (via a man-in-the-middle attack, packet sniffing on an unsecured network, or server log exposure) can decode the credentials instantly using any online tool.

Furthermore, Basic Auth has no concept of session management, token expiry, or permission scopes. There is no way to invalidate credentials without changing the password for all users. This makes it completely unsuitable for multi-user systems or any API exposed to the internet.

Recommended Alternative 1: JWT (JSON Web Tokens)

JWT is a stateless authentication mechanism. The user logs in once with their credentials, and the server responds with a signed token that encodes the user's identity and an expiry time. The client stores this token and sends it in the Authorization header as a Bearer token. The server verifies the token's signature on each request — no password is ever transmitted again. Tokens automatically expire, limiting the damage if one is stolen.

Recommended Alternative 2: OAuth 2.0

OAuth 2.0 is the industry standard for API authorization. It separates authentication (proving who you are) from authorization (what you are allowed to do). It supports fine-grained permission scopes (e.g. read-only vs read-write), refresh tokens, and token revocation. It is used by all major platforms including Google, GitHub, and MTN's own developer APIs. For a mobile money API like MoMo, OAuth 2.0 would be the appropriate choice in a production environment.
